CdPSup	Examen Final	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : ARCHITECTURE ET PROGRAMMATION EN ASSEMBLEUR	Classe : B2 (IT)

Instructions Générales

- Lisez attentivement chaque question avant de répondre.
- Respectez les consignes spécifiques données pour chaque exercice.
- Les programmes doivent être écrits en syntaxe NASM pour l'architecture x86_64.
- La rigueur syntaxique, la clarté, la lisibilité du code et les commentaires sont pris en compte dans la notation.

Partie I — Questions de compréhension (10 points)

1. (1 pt) Expliquez trois différences fondamentales entre un microprocesseur et un microcontrôleur. Donnez un exemple d'utilisation pour chacun.

2. (1 pt) Que sont les registres dans l'architecture AVR ? Citez deux registres typiques et décrivez leur rôle.

3. (1 pt) Quelle est l'utilité de la mémoire Flash dans un microcontrôleur ? Comparez-la à la SRAM et à l'EEPROM.

4. (1 pt) Quelle est la différence entre un assembleur et un compilateur ? Donnez deux exemples de cas d'usage pour chacun.

5. (1 pt) Décrivez les différentes sections d'un programme NASM (`.data`, `.text`, `.bss`). À quoi servent-elles ?

6. (1 pt) En assembleur x64, que signifie l'instruction suivante : `mov rax, [rbx]` ? Expliquez clairement le type d'adressage utilisé.

7. (1 pt) Que représente le registre `RIP` dans l'architecture x64 ? Pourquoi est-il crucial dans le déroulement d'un programme ?

8. (1 pt) Expliquez ce que sont les drapeaux (`flags`) dans un processeur x86/x64. Donnez deux exemples de drapeaux et leur rôle.

9. (1 pt) Quelles sont les étapes principales pour compiler un programme NASM sous Windows (MinGW) ? Citez deux commandes essentielles.

10. (1 pt) Quelle est la différence entre un adressage immédiat, direct, indirect, et indexé en assembleur ? Donnez un exemple pour chacun.

Partie II — Exercices pratiques (10 points)

Exercice 1 : Opérations arithmétiques de base (2 pts)

Complétez un programme NASM x64 qui :

- stocke la valeur 15 dans `RAX`,
- stocke 5 dans `RBX`,
- effectue la division entière `RAX / RBX` (quotient),
- stocke le résultat dans `RCX`.

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Exercice 2 : Utilisation de données en mémoire (2 pts)

Écrivez un programme NASM qui :

- déclare une variable `valeur` = 30,
- charge cette valeur dans `RAX`,
- effectue une multiplication par 3,
- stocke le résultat dans une nouvelle variable `resultat`.

Exercice 3 : Correction de code NASM (2 pts)

Corrigez le programme suivant pour qu'il compile et fonctionne correctement sous NASM + gcc :

```
section .text
global main
main:
    mov rax = 5
    mov rbx, 3
    add rax rbx
    ret
```

Exercice 4 : Transfert registre mémoire (2 pts)

Complétez le programme suivant pour :

- stocker la valeur 100 dans RAX,
- sauvegarder cette valeur dans une variable appelée `memo`,
- relire la valeur dans `RBX`.

```
section .data
    memo     dq 0              ; Réserve mémoire pour stocker la valeur

section .text
global main
main:
    mov     ___, ___           ; Charger 100 dans RAX
    mov     _____, ___        ; Sauvegarde en mémoire
    mov     ___, ______        ; Chargement de la mémoire dans RBX
    ret
```

Exercice 5 : Appel système (2 pts)

Complétez ce programme pour qu'il retourne un code de sortie 0 au système.

```
section .text
global main
main:
    mov     ___, 60     ; Code de l'appel système 'exit' sous Linux
    xor     ___, ___    ; Code de retour = 0
    syscall             ; Appel système
```

NB : Pour Windows, utiliser 'ret' uniquement si aucune interaction système n'est nécessaire.

"Le secret du succès est de faire les choses communes exceptionnellement bien." John D. Rockefeller

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

CdPSup	Corrigé-type de l'Examen Final (Session de Rattrapage)	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : ARCHITECTURE ET PROGRAMMATION EN ASSEMBLEUR	Classe : B2 (IT)

Partie I — Questions de compréhension (10 points)

1. **(1 pt) Trois différences fondamentales entre un microprocesseur et un microcontrôleur :**

   Un microprocesseur est une unité centrale de traitement (CPU) conçue pour exécuter des programmes informatiques, tandis qu'un microcontrôleur est un système complet sur une seule puce, incluant CPU, mémoire, et périphériques.

   - **Architecture :**
     - Microprocesseur : CPU seule, nécessite des composants externes (mémoire, IO).
     - Microcontrôleur : CPU + mémoire (Flash, SRAM, EEPROM) + IO intégrés.
   - **Consommation d'énergie :**
     - Microprocesseur : généralement élevé, adapté aux ordinateurs.
     - Microcontrôleur : faible consommation, adapté aux systèmes embarqués.
   - **Usage typique :**
     - Microprocesseur : PC, serveurs, calcul intensif.
     - Microcontrôleur : objets connectés, robots, appareils ménagers.
   - **Exemples :**
     - Microprocesseur : Intel Core i7, AMD Ryzen.
     - Microcontrôleur : ATmega328 (Arduino Uno), PIC16F877.

2. **(1 pt) Les registres dans l'architecture AVR :**

   Les registres sont des emplacements mémoire rapides utilisés pour stocker temporairement des données et adresses, facilitant l'exécution des instructions.

   - **Registres généraux :** `R0` à `R31`, utilisés pour manipuler données, calculs et échanges.
   - **Registre de statut (`SREG`) :** contient des bits indicateurs d'état (zero, carry, overflow).
   - **Registres spécifiques :**
     - `SP` (Stack Pointer) : pointe vers la pile.
     - `PC` (Program Counter) : adresse de la prochaine instruction.

3. **(1 pt) Utilité de la mémoire Flash et comparaison avec SRAM et EEPROM :**

   - **Flash :** mémoire non volatile utilisée pour stocker le programme exécutable. Elle conserve les données même hors alimentation et est reprogrammable en bloc.
   - **SRAM :** mémoire volatile, très rapide, utilisée pour stocker les variables temporaires pendant l'exécution. Perte de données à l'arrêt.
   - **EEPROM :** mémoire non volatile utilisée pour stocker des données modifiables en cours d'utilisation (ex. : configurations). Moins rapide que Flash, écriture limitée.

4. **(1 pt) Différence entre assembleur et compilateur, avec usages :**

   - **Assembleur :** traduit directement le code assembleur (langage machine lisible) en code binaire exécutable. Utilisé pour contrôle précis du matériel.
   - **Compilateur :** traduit un langage de haut niveau (C, C++, Java) en langage machine, souvent via plusieurs phases d'optimisation.

   Cas d'usage :

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

- Assembleur : programmation bas niveau de périphériques, drivers, ou optimisation critique.
- Compilateur : développement logiciel général, applications desktop, systèmes embarqués complexes.

5. (1 pt) Sections d'un programme NASM :

- `.data` : contient les données initialisées (ex. : chaînes de caractères, variables statiques).
- `.bss` : espace réservé pour les données non initialisées (variables globales allouées en RAM).
- `.text` : section code contenant les instructions exécutables.

Ces sections permettent d'organiser clairement code et données, facilitant le chargement en mémoire et l'exécution.

6. (1 pt) Interprétation de `mov rax, [rbx]` et type d'adressage :

Cette instruction charge dans `rax` la valeur stockée à l'adresse mémoire contenue dans le registre `rbx`.

Type d'adressage : indirect par registre, car l'adresse cible est celle contenue dans `rbx`, ce qui permet un accès dynamique à la mémoire.

7. (1 pt) Rôle du registre `RIP` :

`RIP` (*Instruction Pointer*) contient l'adresse de la prochaine instruction à exécuter.

Il est essentiel pour le flux de contrôle, permettant au processeur de savoir où poursuivre l'exécution, gérer les sauts, appels et retours.

8. (1 pt) Les drapeaux (flags) dans un processeur x86/x64 :

Les flags sont des bits de statut reflétant le résultat des opérations.

- `ZF` (Zero Flag) : activé si le résultat est nul.
- `CF` (Carry Flag) : indique un dépassement en addition ou un emprunt en soustraction.
- `SF` (Sign Flag) : indique le signe du résultat (négatif si 1).
- `OF` (Overflow Flag) : signale un dépassement de capacité pour les entiers signés.

9. (1 pt) Étapes principales pour compiler un programme NASM sous Windows (MinGW) :

(a) **Écriture** du fichier source assembleur (ex. : `prog.asm`).
(b) **Assemblage** en code objet via NASM, par exemple avec `nasm -f win64 prog.asm -o prog.obj`.
(c) **Édition de liens** avec MinGW `gcc` ou `ld` pour créer l'exécutable, ex. : `gcc prog.obj -o prog.exe`.
(d) **Exécution** du programme compilé sous Windows.

Commandes essentielles supplémentaires :

- `nasm -f elf64 prog.asm -o prog.o` (pour Linux)
- `ld prog.o -o prog` (édition de liens directe)

10. (1 pt) Différences entre adressage immédiat, direct, indirect, indexé et exemples :

- **Immédiat :** la donnée est directement dans l'instruction.
  - `mov rax, 10` (charge la valeur 10 dans `rax`)
- **Direct :** l'instruction spécifie explicitement une adresse mémoire.
  - `mov rax, [0x400000]` (charge la valeur à l'adresse 0x400000 dans `rax`)
- **Indirect :** l'adresse mémoire est donnée par un registre.
  - `mov rax, [rbx]` (charge la valeur pointée par `rbx` dans `rax`)
- **Indexé :** calcul de l'adresse avec base + index * facteur + décalage.
  - `mov rax, [rbx + rcx*4 + 8]` (adresse calculée à partir de `rbx`, `rcx` et un offset)

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Partie II — Exercices pratiques (10 points)

Exercice 1 : Opérations arithmétiques de base (2 pts)

Cet exercice évalue la capacité à manipuler des registres de base en assembleur NASM x64 ainsi qu'à effectuer une division entière avec gestion correcte des registres impliqués.

Objectif :

- Mettre 15 dans `RAX`,
- Mettre 5 dans `RBX`,
- Faire la division entière : quotient de `RAX / RBX`,
- Copier le résultat dans `RCX`.

Correction détaillée :

```
section .text
global main
main:
    ; Initialisation des registres
    mov     rax, 15          ; Charger 15 dans le registre RAX
    mov     rbx, 5           ; Charger 5 dans RBX

    ; Avant une division, le registre RDX doit être mis à zéro.
    ; La division 64 bits utilise implicitement RDX:RAX comme dividende.
    xor     rdx, rdx         ; Mise à zéro de RDX pour éviter un reste indésirable

    ; La division div rbx effectue (RDX:RAX) / RBX
    div     rbx              ; Quotient → RAX, Reste → RDX

    ; On copie le quotient dans RCX
    mov     rcx, rax         ; Transfert du résultat
    ret                      ; Fin du programme
```

Exercice 2 : Utilisation de données en mémoire (2 pts)

Cet exercice teste l'accès à des données statiques via les sections `.data`, ainsi que la capacité à modifier une valeur et l'enregistrer.

Objectif :

- Créer une variable `valeur` = 30,
- La charger dans `RAX`,
- Multiplier par 3,
- Sauvegarder le résultat dans une autre variable `resultat`.

Correction détaillée :

```
section .data
    valeur    dq 30          ; Déclaration d'une constante (64 bits)
    resultat  dq 0           ; Espace mémoire réservé pour le résultat

section .text
global main
main:
```


Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

```
    mov     rax, [valeur]      ; Chargement indirect de la valeur (30) en RAX
    imul    rax, 3             ; Multiplication signée : RAX = RAX * 3
    mov     [resultat], rax    ; Stockage du résultat (90) en mémoire
    ret                        ; Fin du programme
```

Note : `imul` permet la multiplication de façon signée. Dans ce cas, cela ne change pas le résultat, mais c'est plus sûr.

Exercice 3 : Correction de code NASM (2 pts)

Code erroné initial :

```
section .text
global main
main:
    mov rax = 5
    mov rbx, 3
    add rax rbx
    ret
```

Erreurs à corriger :

- La syntaxe `mov rax = 5` est incorrecte : NASM n'utilise pas le symbole "=".
- L'instruction `add rax rbx` est invalide : il manque une virgule.

Correction complète :

```
section .text
global main
main:
    mov     rax, 5          ; Affectation correcte
    mov     rbx, 3          ; Initialisation de RBX
    add     rax, rbx        ; Addition correcte : RAX = RAX + RBX
    ret                     ; Fin du programme
```

Exercice 4 : Transfert registre mémoire (2 pts)

Objectif :

- Stocker 100 dans RAX,
- Transférer cette valeur dans une variable mémoire `memo`,
- Recharger cette valeur dans RBX.

Correction détaillée :

```
section .data
    memo     dq 0               ; Variable mémoire 64 bits

section .text
global main
main:
    mov     rax, 100            ; Initialisation de RAX avec 100
    mov     [memo], rax         ; Sauvegarde de RAX dans la mémoire
    mov     rbx, [memo]         ; Rechargement de la mémoire vers RBX
    ret                         ; Fin du programme
```

Note : Ce type d'opération est essentiel pour comprendre les échanges mémoire registre dans l'architecture x86-64.

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Exercice 5 : Appel système (2 pts)

Objectif : Effectuer un appel système standard sous Linux x64 pour terminer proprement un programme avec un code de retour de 0.

Correction :

```
section .text
global main
main:
    mov     rax, 60         ; Code de l'appel système 'exit' sous Linux x86-64
    xor     rdi, rdi        ; Code de retour = 0 (valeur à retourner)
    syscall                 ; Exécution de l'appel système
```

Explication :

- En ABI Linux 64-bit, `rax` contient le numéro de l'appel système.
- `rdi` contient le premier argument : ici, le code de retour.
- `syscall` exécute l'instruction système.

Remarque importante : Sous Windows, cette instruction n'est pas disponible ; on utilise simplement `ret` si aucune interaction système n'est requise.

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net